

Techniques for routeing and scheduling services on a transmission network

B Purohit, T Clark and T Richards

A problem is presented that requires both routeing and scheduling of multiple services on a circuit switched transmission network. The techniques investigated were simulated annealing, classifier systems, hill climbing and random search. Two groups of test problems were used and the preliminary results are presented which provide a ranking of various techniques.

1. Introduction

A given network with a particular topology and links with certain capacities can support a variety of services. It is assumed that requests for services are received, with each service required at a particular time and for a certain duration. The problem involves flexibly allocating required bandwidth to services in order to satisfy as many of the service requests as possible with a minimum cost. The cost is defined in terms of an objective function which considers the path length of allocated services and a penalty for unallocated services. The network routeing/scheduling problem is therefore an optimisation problem. The 'complexity' of the problem can be controlled by the setting of various control parameters such as the number of destination nodes per service, number of service requests, etc. The routeing/scheduling problem is presented more formally in section 2, followed by problem analysis and techniques for solving the problem in section 3. Results of the preliminary study are presented in section 4.

2. The routeing/scheduling problem

2.1 Network tuple

The topology and capacity of the network is described by the tuple: (N, E, cap) where:

N is the set of nodes,

E is the set of edges or connecting links.

link : $N \times N \rightarrow E$ Given two nodes, returns the link that connects them — note that link $(N1, N2)$ may not equal link $(N2, N1)$,

cap : $E \rightarrow C$ is a function that maps a link on to a member of the set, $C = \{c_1, c_2, c_3, \dots, c_n\}$ where c_i is a valid link capacity.

2.2 Service tuple

Services are modelled by the tuple: $ST_i = (S_i, D_i, Dur_i, B_i, P_i, Start_i)$ where:

S_i is the source node, i.e. $S_i \in N$,

D_i is the set of destination nodes, $D_i = \{D_{i,1}, D_{i,2}, \dots, D_{i,1}\}$ where $D_{ij} \in N$ (when the set contains only one element, the service is point to point; when the set contains several elements, the service is point to many points, i.e. a broadcast),

B_i is the required bandwidth of the service, $B_i \in \mathbf{Z}$, i.e. the set of positive integers,

P_i is the service priority, $P_i \in \mathbf{R}$, i.e. the set of positive real numbers,

Dur_i is the duration of the service, $Dur_i \in \mathbf{R}$, i.e. the set of positive real numbers,

$Start_i$ is a start time for the service, $Start_i \in \mathbf{R}$, i.e. the set of positive real numbers.

There are many additions that can be made to this basic formulation. For instance it may be the case that there should be link 'separacy' between several services. This means that for reasons of robustness to network failure these services should not share any common links. The work presented does not address these additional issues.

2.3 Scheduler

The scheduler is given a list of service tuples and its purpose is to allocate a path for each service request in this list such that the path is available with required bandwidth at the required time. Before defining a schedule let the following definitions hold:

$path(ST_i)$ will return all of the paths for service instance ST_i — this is noted in the set, $Paths = \{path_1, path_2, \dots, path_m\}$ (if the service is point to point, then only one path will be returned), $path_i^n$ is the n th link member of the path of i , link capacity given by: $cap(path_i^n)$, $|path_i|$ is the length of the path i .

Then, a path is valid if, and only if, $path_i^1 = S_i$ and $path_i^{|path_i|} \in D_j$ and $(cap(path_i^n) \geq B_i \text{ for } n=1 \dots |path_i|)$ for all $i \in Paths$, i.e. the path is valid if the first node is the source node, the last node is the destination node and each of the links has enough capacity to route the service. Also:

$f_t E \rightarrow SI$ is a function that, given a link, E , returns the set of services scheduled to run on that link at time t ,
 $S_{(t)} \cup f_i(i)$ is the allocation of all links to service instances at time t .

A schedule then is:

$S \cup S_{(t)}$ the allocation of all links from t_{now} to some future time T .

As there are potentially many feasible schedules, an objective function is used to assess the 'goodness' of a particular schedule.

2.4 An objective function

The cost of a schedule may be determined by an objective function, an example of which is:

$$OB = \alpha \sum_{i \in S} |path(ST_i)| + \beta \sum_{(j \in STI)} (P_j)$$

It is stressed that this function serves only as an example and more complex objective functions may be necessary.

This has the effect of summing the length of all of the paths of services that have been scheduled and adding to it a penalty (P_j) for all unallocated services. Therefore a low-cost solution would be one that has minimal path length and satisfies every service request. The α and β terms are weights that represent the relative importance of the two constraints. Values of $\alpha=1$, $\beta=100$ and $P_j=1$ were used.

3. Analysis of the problem and techniques for the routeing/scheduling problem

Several techniques were applied to the routeing/ scheduling problem. These include simulated annealing, classifier systems, hill climbing and random search. The architec-

ture of the test-bed is presented (see Fig 1), followed by brief descriptions of classifier systems, simulated annealing and hill climbing.

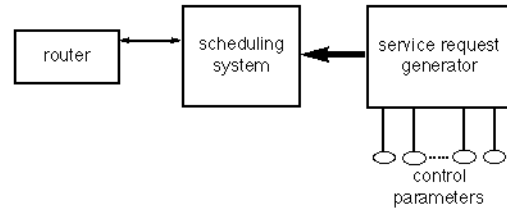


Fig 1 The testbed architecture.

A service-request generator is used to generate dynamically a list of service requests. The operation of the generator is fully controllable through a set of gaussian parameters, e.g. number of services to generate, the number of destinations the services ought to have, etc. The resulting list is then given to the scheduling system, which has a finite time within which to produce a schedule. The generator is invoked again, adding to the existing list of requests. The scheduling system is provided with the updated list and has the opportunity to revise its existing schedule or to reschedule from scratch. The presented problem is therefore an incremental scheduling one. The preliminary study involved just one iteration around this loop, - allowing each of the techniques as much time as was needed to return with a response. All of the techniques made use of a router, written in ECL¹PS^e, which was capable of determining feasible paths for broadcast services ECL¹PS^e, developed by the ECRC, provides a unified environment in which knowledge-base and constraint-logic programming technologies are integrated.

The next section gives an analysis of the problems generated by the service-request generator.

3.1 Problem analysis

The cost function provides a method for comparing the performance of different scheduling techniques. In addition to this it is necessary to know the qualitative nature of the problem to which the scheduling technique has been applied, e.g. whether or not the particular problem is under- or over-resourced.

Some analysis was performed on sample problems produced by the problem generator.

The method of investigation was to identify upper and lower limits on each of the control parameters. These control parameters are:

- scheduling window length-service must start and complete within this time window,

- service duration — the duration of the service expressed as a percentage of the time window length,
- reaction time of the scheduler—the length of time before the scheduler must produce a schedule for the services,
- number of services,
- number of destinations per service,
- total capacity requested by a block of services,
- distribution of the total capacity between the block of services.

Lining up these parameters, one per axis, gives a hypercube with each point within the cube representing a problem. It is not feasible to examine a significant portion of the points within the hypercube owing to the enormous amount of computational power needed to do this. However, extrema were investigated, and qualitative statements about these problems can be made. It is suspected, though not proven, that as trajectories are traced through the cube the problem difficulty changes with perhaps sharp transition points very much like phase transitions [1]. The extrema of the hypercube were examined using a random scheduler coupled with the above cost function.

Several of the parameters were fixed for the experiments reported in this paper. In particular, services had fixed durations (30 minutes), fixed number of destinations (4 nodes). The three factors that were varied were the number of service requests to schedule, the total amount of capacity demanded by these requests and the distribution of capacity between the services. The concern was to move from under-resourced problems to overresourced ones.

Figure 2 shows a schematic of the problem types. As at point A there is only a low number of small services, most of which are likely to be routed, the main problem here will be to reduce path length. If the services are small compared to the link capacities then this is an overresourced problem. The problem can be made underresourced in several ways. Either increase the number of services (to get to point B), or increase the amount of capacity demanded by the services (to get to point C) or do both (to get to point D).

In the extreme, over-resourced problems are easy to solve, and it is expected that relatively simple techniques will perform well here. Similarly problems that are extremely under-resourced are easy to solve since little or none of the requests can be met. The interesting points lie between these two extremes (shown as point E and F in Fig 2). Here it is expected that good solutions can only be reached by sophisticated techniques. The front face of the cube (dotted line) defines a set of problems analogous to the back face, except that there is greater variability in the

distribution of capacity between the services. This is akin to adding noise to the problem.

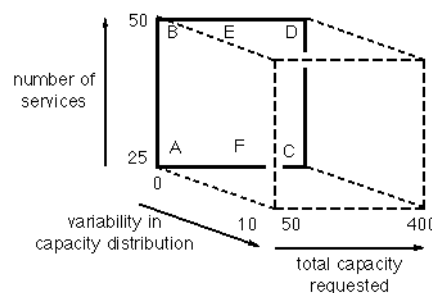


Fig 2 The problem hypercube.

For these experiments values of 25 and 50 were taken for the number of services and values of 50, 100, 200, 300 and 400 for the total service capacity and variability values of 0.0 (equal capacities) and 10.0 (highly unequal capacities) for the capacity distribution.

The techniques applied to this problem are explained next, beginning with the classifier system.

3.2 A classifier for the problem

The classifier system (see Fig 3) is an adaptive rulebased system that uses genetic algorithms and reinforcement learning methods to alter its behaviour [2]. They have been used for a variety of tasks and applications of particular interest in this problem include job-shop scheduling [3] and load balancing in distributed computer systems [4].

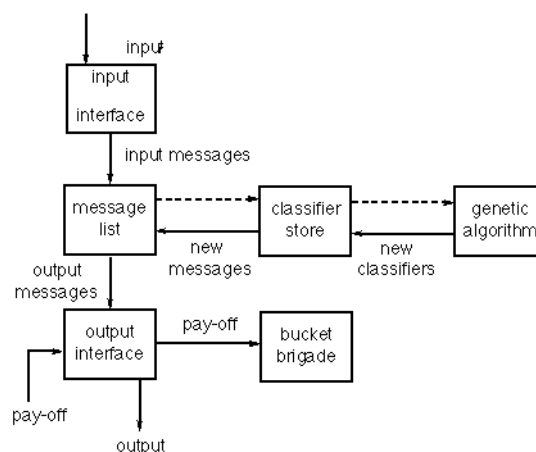


Fig 3 Architecture of classifier systems.

There are three main components of a classifier system - a set of rules, a rule-discovery algorithm and a reinforcement learning algorithm. The classifier rules are condition-action rules constructed over a trinary alphabet. A classifier system's interaction with its environment is carried out through binary strings that are interpreted and produced by the classifier rules. Each classifier rule has a strength value that is used by the rule-discovery and reinforcement mechanisms. The rule-discovery algorithm is usually a genetic algorithm which is periodically applied to the rule set to produce a new set of rules. The reinforcement mechanism used in a classifier system is known as the bucket-brigade algorithm. This is responsible for distributing reward amongst the classifier rules so as to form chains of rules that achieve a particular goal.

The classifier system used for this task is acting as an informed permutation generator. It manipulates a list of service requests according to their content. This is achieved by the use of classifier rules that prioritise requests according to their content.

The initial system is very simple, attending only to the bandwidth of a set of requests. A service's bandwidth can be mapped into one of three possible regions based upon the distribution of link capacity within the network. There are three regions-low, medium and high-and these are mapped on to two bits of a classifier rule using a Grey coding. Grey coding maps values into distinct regions of the binary digit space, i.e. high values will have similar codes that are dissimilar to the codes for lower values. This is unlike a traditional binary coding where two vastly different numbers may only have a one bit difference between their codes. This makes pattern matching in classifier systems easier, in that a classifier rule can match a continuous range of values instead of a fragmented set of values across the parameter space.

The system's output consists of a priority value for a particular service that is used to reorder the list of service requests. To enable a classifier rule to refer to a specific service request, all output messages have been tagged with the identity of the service that activated them. The content of the output message is a priority value encoded as a seven-bit binary string. The pass-through property of classifier rules is not being used on this problem and all rules are single-condition, single-action rules.

The system currently only acts as a stimulus-response system, i.e. no internal messages are being generated or used by the classifier rules. A population of fifty classifier rules is used, with a genetic algorithm being applied every 100 iterations to the rules. The bucket-brigade algorithm is not applied.

The system works by firstly translating all service requests into classifier messages. The classifier rules then respond to these messages with a set of output messages that prioritise the service requests. Conflicts in a service's priority are resolved by taking the priority value with the most support.

3.3 A hill climber for the problem

The hill climber used the algorithm depicted in Fig 4. The algorithm terminates when the hill climber has settled into a local minima. At which point the search is repeated again from another random point. The representation and move operator used for the hill climber was the same as that for the simulated annealer (described below) except that the hill climber used a fixed rather than variable neighbourhood size.

```

START
current_soln=random_soln()
for j=1 to number_of_neighbours
  neighbourj=move(current_soln)
  if (cost(neighbourj)<cost(current_soln))
    then (current_soln=neighbourj; j=0)
  endfor j
return current_soln
STOP

```

Fig 4 Hill climbing algorithm.

The number of neighbours is given by:

$$N = \frac{L(L-1)}{2}$$

where L is the number of service requests.

3.4 A simulated annealer for the problem

The simulated annealing algorithm for the routeing/scheduling problem is shown in Fig 5. The algorithm differs from traditional hill climbing in that it allows nonimproving moves to be accepted with a probability dependent on the degree of non-improvement and a control parameter, termed the temperature. More specifically the acceptance criteria is:

$$p(x) = e^{\frac{-\Delta}{T}}$$

where Δ is defined as the difference between the current cost and the neighbour's cost and T is the control temperature.

Initially the temperature is set high, giving a good chance of accepting non-improving moves. The temperature is reduced gradually. At high temperatures the annealer acts very much like random search and at low temperatures it is effectively a hill climber. The rationale for the temperature parameter is to allow the annealer to avoid or escape local minima and yet still settle into good solutions. The

effectiveness of the annealer is very much dependent upon the initial temperature, the time spent at each particular temperature and the rate at which cooling occurs. This is collectively known as the annealing schedule. For these experiments suitable values were 500 for the initial temperature, 0.9 for the cooling rate and 20 iterations at each temperature step.

```

START
temp=init_temp;
current_soln=random_soln()
best_soln=current_soln
for i=1 to num_iterations
  for j=1 to iterations_per_temp
    neighbour=move(current_soln)
    delta=cost(neighbour)-cost(current_soln)
    if (cost(neighbour)<cost(current_soln)) OR
      acceptance_criteria(delta,temp)
    then current_soln=neighbour
    if cost(current_soln)<cost(best_soln) then
      best_soln=current_soln
    endfor j
  temp=temp*reduction_factor
end for i
return best_soln
STOP

move(current_soln)
START
for i=1 to [num_requests*(temp/init_temp)] + 1
  swap(random(1,num_requests),
    random(1,num_requests))
end for i
STOP

```

Fig 5 Simulated annealing algorithm.

The representation used for the annealer, the hill climber and the random search was to consider a list of requests as a permutation to be worked on (see Fig 6). The annealer was supplied with a move operator that swapped two elements in the current permutation to yield a new permutation (as shown in Fig 6). Note that the number of swaps carried out by the annealer is a function of the temperature. Initially this is high, giving a large neighbourhood and allowing large movements in the search space to be taken. As the temperature is reduced, so is the size of neighbourhood. This has proved to be more effective than using a constant neighbourhood size.

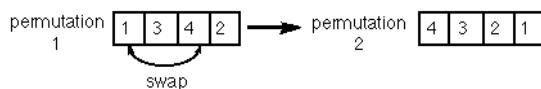


Fig 6 Move operator.

Each permutation is costed by the router that takes the permutation and attempts to route them in the specified order. Thus in permutation 1, the network would be loaded with service request 1 first, followed by service request 3 then 4 then 2. The cost that is returned is dependent upon the order in which the services are considered for routing; different permutations will therefore yield different costs.

The annealer, then, is working to find the optimal permutation. The router is described below.

3.5 The router used to evaluate permutations

Given a service request the router must find a valid path for the request if such a path exists. It is desirable for the router to yield as short a feasible path for the service request as possible, and different search strategies and heuristics may be used to solve this routeing problem. Several different routeing algorithms were implemented and compared for performance. These algorithms are briefly outlined below.

- A heuristic algorithm

Given that the nodes already in the path so far are $\{N_1, N_2, \dots, N_m\}$ then an estimate for the total cost of this path once it is extended to a solution path is:

$$(|\{N_1, N_2, \dots, N_m\}| - 1 + \max_dist(\{N_1, N_2, \dots, N_m\}, \{UnvisitedDest\}))$$

The 'unvisited destinations' are those in the request not contained in $\{N_1, N_2, \dots, N_m\}$. Also:

$$\begin{aligned} \max_dist(\text{Set}, \{X_1, X_2, \dots, X_n\}) &= \\ &\max(\max_dist(\text{Set}, X_1), \dots, \max_dist(\text{Set}, X_n), \\ &\min_dist(\text{Set}, X_1), \dots, \min_dist(\text{Set}, X_n)) \\ \text{and } \min_dist(\{N_1, N_2, \dots, N_m\}, X) &= \\ &\min(\text{dist}(N_1, X), \dots, \text{dist}(N_m, X)) \end{aligned}$$

For the nodes X and Y , $\text{dist}(X, Y)$ is the shortest distance between X and Y on the network if the network is empty. This is precompiled information generated as soon as the network topology is known and remains consistent as long as the topology remains unchanged. The network used for this paper had 30 nodes and 69 directed arcs, and it took 14 seconds for ECL^{PS} techniques to generate this information. This is an underestimate and so is a valid heuristic for best-first search.

- A partitioning routeing algorithm

This is a method for avoiding redundant search. Figure 7 shows a small directed graph. Suppose that a service request requires node 1 as source and has node 7 as a single destination. Then the first choice is either to connect 1 to 2 or 3. If node 3 is selected, then there is no choice but to connect 3 to 4. Now there is again a choice to connect 4 to 2 or 6 or 7 or 5. If node 2 is selected then it is impossible to get to destination 7 without looping. Therefore node 2 is removed from the set of choices.

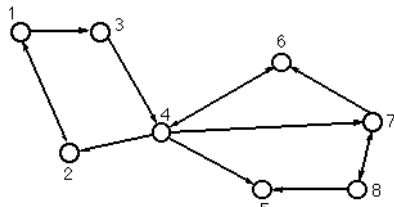


Fig 7 An example network.

- A destination-ordering algorithm

Destinations are ordered by furthest destination first (using precompiled data). This means that destinations are connected to the source node by the furthest first order. There is a possibility that, if an accurate heuristic is used, some of the destinations will be connected automatically if the furthest destinations are connected first.

The results shown in Fig 8 compare eight different search procedures. Four are based on depth-first search (DF) with various heuristics, partitioning (P) and ordering or destination (O). The best-first (BF) search requires the distance measures as a heuristic, although this may be built into depth-first, though no speed-up was noted.

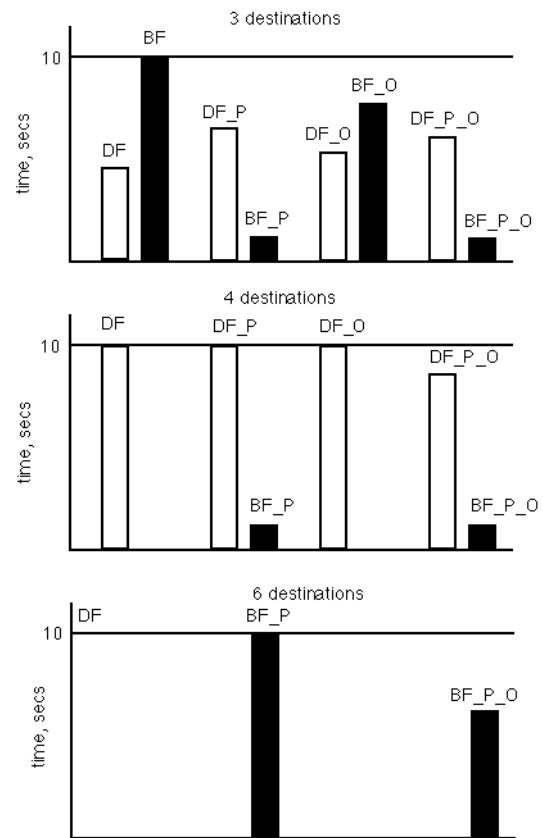


Fig 8 Router results.

The three graphs in Fig 8 compare the eight algorithms averaged over ten problems using three destinations, another ten problems with four destinations and ten with six destinations. Only BF_P (best-first with partitioning) and BF_P_O (best-first with partitioning and ordering of destinations) could route with at least size destinations within 20 seconds. Each of the algorithms returned with comparable cost (i.e. path length). However, the concern here was with the speed of the routing algorithms, since they were to be used intensively by iterative techniques for scheduling.

4. Experiments and results

The extrema of the hypercube in Fig 2 as well as several intermediate points were investigated. Each of the techniques was allowed unlimited time to return with an answer. The results are shown in Table 1 (for the back face of the cube) and Table 2 (for the front face), as well as graphically in Fig 9. The rows show that twenty-five and fifty services were used. The columns show the total amount of capacity for the services. The services have equal capacities in Table 1 and unequal capacities in Table 2. Each entry is of the form:

number_of_services_routed (cost of solution)

where cost of solution is as specified in the earlier cost function. The entries are ordered as:

R random search

C classifier system

S simulated annealer

H hill climber

The bold entry shows the highest ranking technique for that problem (based on the number of services routed). Each entry is the rounded, averaged value of 50 runs on 50 different service tuples. The intention was to present a spectrum of problems to the scheduling techniques, ranging from over-resourced problems to under-resourced problems. The problems on the left side of the table are over-resourced problems and as you move left to right they gradually become under-resourced. On the graphic presentation in Fig 9, the y axis shows the number of service requests and the x axis shows four problems (involving 50, 100, 200, 300 and 400 units of capacity).

Table 1 Equal capacities.

Var = 0.0		50	100	200	300	400
25 services	R	21 (995)	12 (1763)	4 (3472)	5 (4036)	2 (4424)
	C	22 (1157)	14 (3281)	7 (5491)	4 (6363)	3 (6743)
	S	21 (995)	17 (1767)	8 (3469)	5 (4048)	3 (4425)
	H	23 (614)	14 (1375)	6 (3467)	5 (4043)	4 (4231)
50 services	R	48 (835)	33 (2751)	18 (6369)	9 (7518)	8 (8273)
	C	47 (1466)	34 (5256)	16 (8974)	10 (9046)	8 (9385)
	S	45 (1431)	35 (3334)	18 (6563)	13 (7259)	10 (8075)
	H	49 (650)	31 (1406)	23 (5595)	12 (4816)	9 (5614)

Table 2 Unequal capacities.

Var = 10.0		50	100	200	300	400
25 services	R	23 (611)	19 (1183)	12 (1785)	9 (3091)	9 (3477)
	C	23 (782)	20 (1624)	15 (3225)	10 (4706)	7 (5407)
	S	23 (614)	22 (809)	17 (1759)	10 (3089)	6 (3854)
	H	22 (803)	19 (1183)	15 (3552)	11 (2902)	8 (3483)
50 services	R	47 (1009)	36 (2001)	24 (3712)	21 (4673)	17 (5430)
	C	47 (946)	44 (1992)	37 (4233)	31 (5980)	26 (7578)
	S	45 (1392)	34 (3515)	41 (2170)	28 (4656)	23 (5619)
	H	48 (835)	45 (1407)	41 (2173)	33 (3314)	31 (4078)

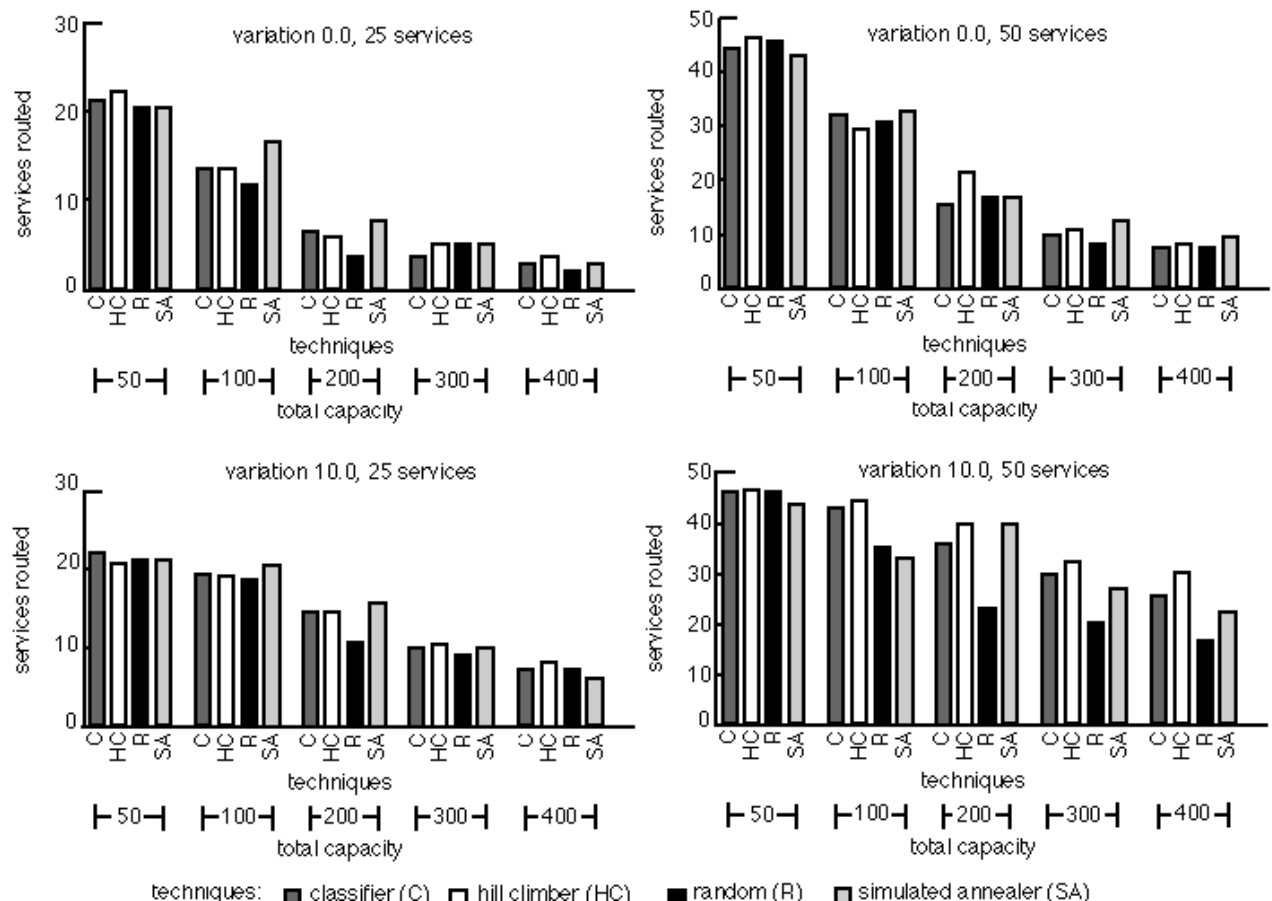


Fig 9 Graphical representation of Tables 1 and 2.

The results show that, on the most over-resourced problems presented, the average number of service requests satisfied range from 87% to 94% (see Table 3). It should be noted that problems have not been presented for which all of the service requests can be satisfied. For the over-resourced problems the results show that generally random search is often better than either simulated annealing or hill climbing, but at the same time worse than one of these, e.g. in 0.0_50_50¹, random search does better than hill climbing but worse than annealing, and in 10.0_50_50, random search does better than annealing but worse than hill climbing.

At the other end of the spectrum, for under-resourced problems, Table 4 shows that between 17% and 48% of service requests are still satisfied. It would have been desirable to present more severely under-resourced problems with about 5% or less of the service requests satisfied. This shows that the problems have not been made sufficiently under-resourced. The results for the under-resourced problems indicate that, generally, random search is worse than annealing or hill climbing, e.g. in 0.0_25_400, 0.0_50_100, 10.0_50_500, and 10.0_25_400 random search performs worse than annealing or hill climbing.

It would appear from these results, and from those reported on the analysis of problem difficulty [5], that the problems on the left hand sides of Tables 1 and 2 may not be soluble, i.e. there may be no solutions that satisfy all of the service requests. Likewise, problems at the right hand end of these tables cannot be easily proved as insoluble, i.e. none of the service requests can be satisfied. The spectrum of problems can therefore be seen to lie between these values, in a hard region (referred to as a 'mush' region in Smith [5]). This may account for why random search performs as well as annealing or hill climbing in the 'under-resourced' problems. All of the results presented here are consistent with other reported studies that, compare, these scheduling techniques and have also found these hard regions [6].

The results obtained allow a ranking to be performed, based on how many cases or problems the various search techniques 'won'. Simulated annealing wins 5 out of 10 cases (from Table 1) and the hill climber wins 4 out of 10 from the same table. Random search wins 1 case and the classifier wins none. On this basis it is possible to rank the techniques in order for Table 1 as: simulated annealing, hill climber, random search and classifier system. On the same basis, the ranking for Table 2 can be set as hill climber, simulated annealing, random search and classifier system.

It is also noteworthy that, generally, more service requests are satisfiable when they have unequal capacities, i.e. more services are satisfied in Table 2 than in Table 1.

5. Conclusions and future work

Improvements to the reported scheduling techniques can be made with more elaborate versions of each of these techniques. These results form a benchmark against which to test variations of simulated annealing and classifier systems. A possible area of change to be considered more closely for the annealer is the neighbourhood operator. Following Glover [7] a non-permutation operator will be investigated to determine whether or not the underlying problem is one of sequencing. The operator could work in the following manner. The router is required to return not just a single path for a particular service request but several. The annealer may then have operators to examine the paths of service requests and allocate them an alternative path. Alternatively, the router is asked to alter some part of the path. This method contrasts with the existing annealer in that it locates more of the work in the annealer rather than in the router and may provide for faster responses, since the router is relatively slow. Domain specific knowledge may be used to determine which paths to alter and how. In general, domain-specific operators are likely to produce better results than more general syntactic operators such as swap (a,b).

¹ Table entries are referred to in a short hand. An entry of the form '0.0_50_50' refers to variation of 0.0 (Table 1), 50 services (row 2) and total capacity of 50 units (1st column)

Table 3 Over-resourced network — average number of services satisfied.

Variation = 0.0	50 units of cap
25 services	21.75 (87%)
50 services	47.25 (97.5%)

Variation = 0.0	50 units of cap
25 services	22.75 (97%)
50 services	46.75 (93%)

Table 4 Under-resourced network — average number of services satisfied.

Variation = 10.0	400 units of cap
25 services	6 (24%)
50 services	8.7 (17%)

Variation = 10.0	400 units of cap
25 services	7 (28%)
50 services	24.2 (48%)

There is also the possibility to use a divide-and-conquer approach to the scheduling problem. The benefit being that the problem may then be solved in parallel, providing for possibly faster responses. The problem might be passed to a routine that attempts to determine temporal dependencies between the service requests. As an example consider the services in Fig 10. Services a, b and c are either directly or indirectly dependent on each other (two services must be directly dependent on each other if their time windows overlap). However, they are completely independent of services d, e and f. Similarly services d and e are dependent on each other and not dependent on any other service. It should also be noted that service f is independent of any other service and therefore can be removed from the list of service permutations (thus reducing the size of the search space). The remaining services can be split into two groups-(a, b and c) and (d and e). Each of these can be solved separately using multiple schedulers and the results returned can be patched together quite easily.

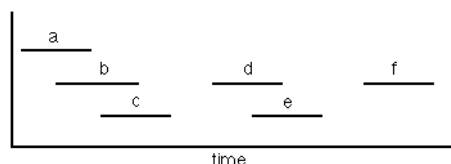


Fig 10 Exploiting temporal dependencies.

It would be desirable to also detect and exploit resource dependencies in a similar manner, but these dependencies can only be guessed at rather than being known for certain.

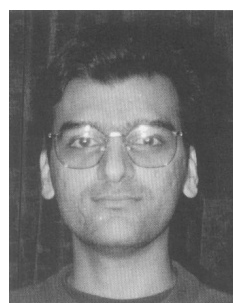
The classifier system could be improved by providing it with information about the different types of service requests that are present, and the proportion in which they are present. A more complex rule syntax is needed perhaps, with the addition of a better representation scheme that is more able to cope with relational and numerical rules than the traditional binary representation [8]. More complex classifier systems are being investigated as well as methods for dividing the problem between multiple classifier systems that are either part of a centralised or a distributed scheduling scheme.

Acknowledgements

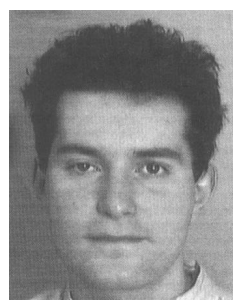
The authors wish to acknowledge the assistance of C Muller, B Crabtree and N Azarmi in the preparation of this paper.

References

- 1 Prosser P: 'An empirical study of phase transitions in binary constraint satisfaction problems', University of Strathclyde Technical Report AISL-49-93 (1993).
- 2 Booker L B, Goldberg D E and Holland J H: 'Classifier systems and genetic algorithms', *Artificial Intelligence*, **40**, pp 235—282 (1989).
- 3 Hilliard M R, Liepins G E, Palmer M, Morrow M and Richardson J: 'A classifier-based system for discovering scheduling heuristics', in Grefenstette J J (Ed): 'Proceedings of the Second International Conference on Genetic Algorithms', pp 231—236 (1987).
- 4 Sannier A V and Goodman-Midgard E D: 'A genetic approach to adaptive load balancing for distributed systems', in Laird J E (Ed): 'Proceedings of the Fifth International Conference on Machine Learning', pp 174—180 (1988).
- 5 Smith B: 'Phase transitions and the mushy region in constraint satisfaction problems', in ECAI (1994).
- 6 Brind C, Muller C and Prosser P: 'A study of stochastic search techniques applied to vehicle routing problems-parts I and 2', Centre for Resource Management, University of Strathclyde (June 1994).
- 7 Glover F: 'TABU search for a bandwidth packing problem', *Journal on Computing*, **1** (1989).
- 8 Shu L and Schaeffer J: 'VCS: variable classifier systems', in Schaeffer J D (Ed): 'Proceedings of the Third International Conference on Genetic Algorithms', pp 334—339 (1989).

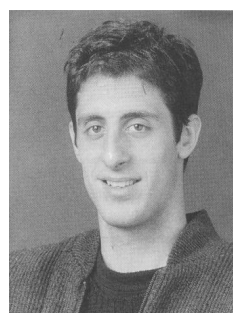


Bharat Purohit graduated from Teesside University in 1988 with a degree in Computer Science. Since joining BT in 1989, he has worked in a variety of areas developing prototypes for fault correlation, detection of congestion on networks, and intelligent hypertext systems. His interests include neural computing, evolutionary computing and stochastic search methods. He is currently studying for a doctorate in neural computing at King's College London.



Tim Clark obtained a BEng in Electronic and Electrical Engineering from University College of Swansea in 1992.

He is currently studying for a PhD in the field of genetic-based learning and distributed systems at the University of Wales, Swansea; this work is sponsored by BT Laboratories.



Tom Richards received a BSc in mathematics in 1991 from University of London and his MSc in mathematics in 1992 also from University of London. He has been studying for a BT sponsored PhD degree at IC-Parc (Imperial College).

His research work covers techniques for constraints satisfaction and optimisation problems.